



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 9.2

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 05 de mayo del 2026

CALIFICACIÓN: _____

ACTIVIDADES REALIZADAS

ACTIVIDAD 1. Humo

Instrucciones: “*Editar Textura para el humo, Blending para el humo, Animación del humo: no sale al inicio, se calienta el Aeolipile y empieza a salir, crece después de un tiempo y al apagarse el fuego el humo va decreciendo hasta ya no salir.*”

Nos basamos en nuestro ejercicio anterior para esta práctica, los cambios en esta práctica solo se dieron dentro de nuestro archivo main.

Comenzamos exportando nuestra textura e instanciando tres variables para la animación del mismo.

```
Texture Humo;
// --- Animación: Efectos del Humo ---
float escalaHumo;      // Factor de tamaño dinámico que calculamos a partir de velAeolipile
float ondulacionX;     // Desplazamiento flotante en X usando la función seno
float ondulacionZ;     // Desplazamiento flotante en Z usando la función coseno
...
...
Humo = Texture("Textures/humo1.png");      Humo.LoadTextureA();
// --- Valores Iniciales: Efectos del Humo ---
escalaHumo = 0.0f;      // Inicia invisible
ondulacionX = 0.0f;     // Inicia sin ondulación
ondulacionZ = 0.0f;     // Inicia sin ondulación
```

Para controlar el ciclo de vida de nuestro humo, vinculamos el cuándo se ve (su existencia) y escala directamente el movimiento de la esfera del Aeolipile con la variable velAeolipile. Definimos que, al encenderse el fuego, la velocidad aumente progresivamente hasta un tope de 15.0f, lo que usamos más adelante como un indicador para que el humo empiece a salir y crezca en sus escalas. La modificación importante sucede en la Etapa 4 de nuestra lógica pasada de rotación del Aeolipile, en el momento de frenado, una vez que el fuego se apaga, programamos una desaceleración gradual ($\text{velAeolipile} -= 0.05f * \text{deltaTime}$). Al ser el tamaño del humo directamente proporcional a esta velocidad, esta resta constante genera el efecto visual de que el humo va haciéndose más pequeño hasta desaparecer cuando el Aeolipile se detiene por completo.

```
// 6. Lógica de Rotación del Aeolipile -----
if (mainWindow.getAnimacionFuego()) {
    tiempoFuego += deltaTime; // Inicia el cronómetro del fuego

    // ETAPA 1 y 2: CALENTAMIENTO Y ACELERACIÓN
    // Inicia después de 125.0f (2.5 seg)
    // -> Acelera hasta llegar al tope en 625.0f (+10 seg)
    // -> Se queda en el tope hasta los 875.0f (+5 seg)
    if (tiempoFuego >= 125.0f && tiempoFuego < 875.0f) {
        tiempoGirando = tiempoFuego - 125.0f;
        // ARRANQUE SUAVE: La velocidad sube progresivamente
        velAeolipile = tiempoGirando * 0.03f;
        // TOPE DE SEGURIDAD: Límite máximo de velocidad para el motor
        if (velAeolipile > 15.0f) {
            velAeolipile = 15.0f;
        }
    }

    // ETAPA 3: VELOCIDAD PUNTA Y EXTINCIÓN
    // A partir de los 875.0f (17.5 seg?), el fuego comienza a extinguirse
    // -> Durante estos últimos 5 segundos de fuego, el motor mantiene su movimiento
    else if (tiempoFuego >= 875.0f) {
        // MIENTRAS EL FUEGO SE HACE CHICO, EL MOTOR SIGUE A MAX VEL
        velAeolipile = 15.0f;
    }
}
}
```

```

else {
    // ETAPA 4: FRENADO POR "FRICCIÓN"
    // CUANDO EL FUEGO YA DESAPARECIÓ POR COMPLETO Y EL INTERRUPTOR SE APAGÓ
    if (velAeolipile > 0.0f) {
        // FRENADO LENTO: Pierde vel gradualmente durante 300.0f unidades (6 seg).
        // -> Al restar esta velocidad, el humo sincronizado también irá desapareciendo.
        velAeolipile -= 0.05f * deltaTime;

        // Tope de seguridad para evitar que el motor empiece a girar en reversa
        if (velAeolipile < 0.0f) {
            velAeolipile = 0.0f;
        }
    }
    // ETAPA 5: REPOSO TOTAL Y REINICIO
    else {
        // Si ya apagamos el fuego y el motor se detuvo por completo:
        // -> Regresamos el cronómetro a cero para dejar la máquina lista para otro ciclo.
        tiempoFuego = 0.0f;
        // -> Reiniciamos la bandera para devolver la esfera a su punto de origen.
        esferaLanzada = false; //LA ESFERA REAPARECE EN EL BRAZO
    }
}

// --- APLICACIÓN FINAL DE LA ROTACIÓN ---
// Sumamos la velocidad calculada a los grados totales del objeto
anguloAeolipile += velAeolipile * deltaTime;
// Reset para no llegar a números gigantes y evitar errores de memoria (> 360 grados)
if (anguloAeolipile > 360.0f) {
    anguloAeolipile -= 360.0f;
}
}

```

En nuestra etapa de renderizado de modelos transparentes, condicionamos el renderizado del humo a que el Aeolipile este moviéndose ($velAeolipile > 0.0f$). El requerimiento de que el humo crezca y decrezca se logró calculando un porcentaje dinámico ($escalaHumo$), el cual divide la velocidad actual entre la máxima permitida ($15.0f$) y se aplica directamente a la matriz de escalado. Para la “Animación del humo”, implementamos un efecto de flotación utilizando las funciones del ejercicio pasado con seno y coseno multiplicadas por el tiempo; esto genera una ondulación entre los ejes X y Z que también es proporcional a la velocidad del Aeolipile. Finalmente, aplicamos la textura editada sobre un modelo geométrico de planos cruzados ($meshList[3]$).

```

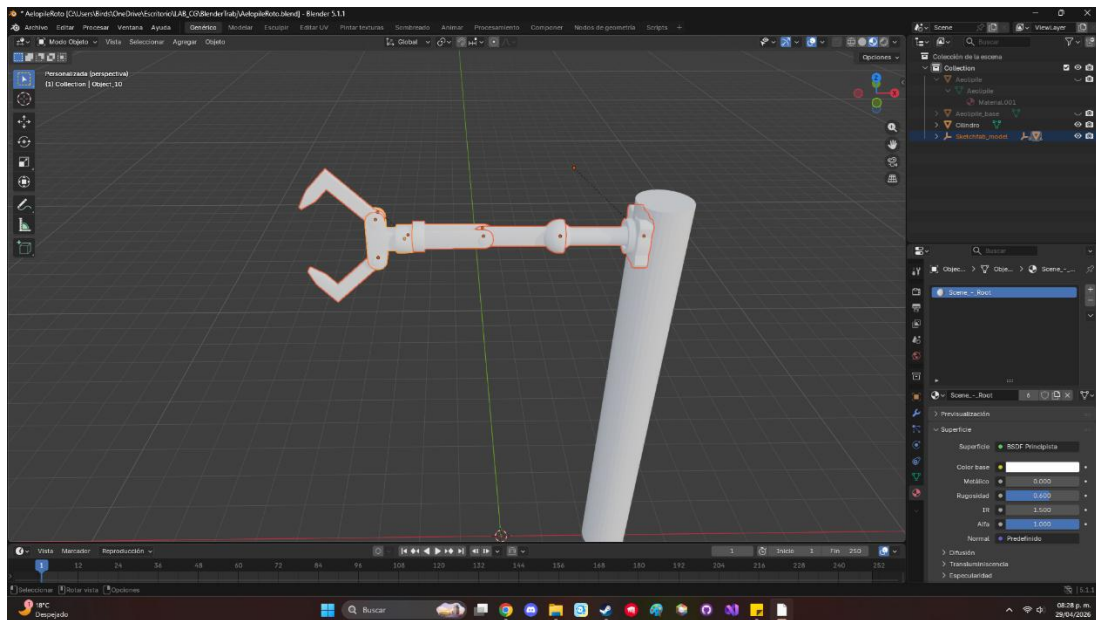
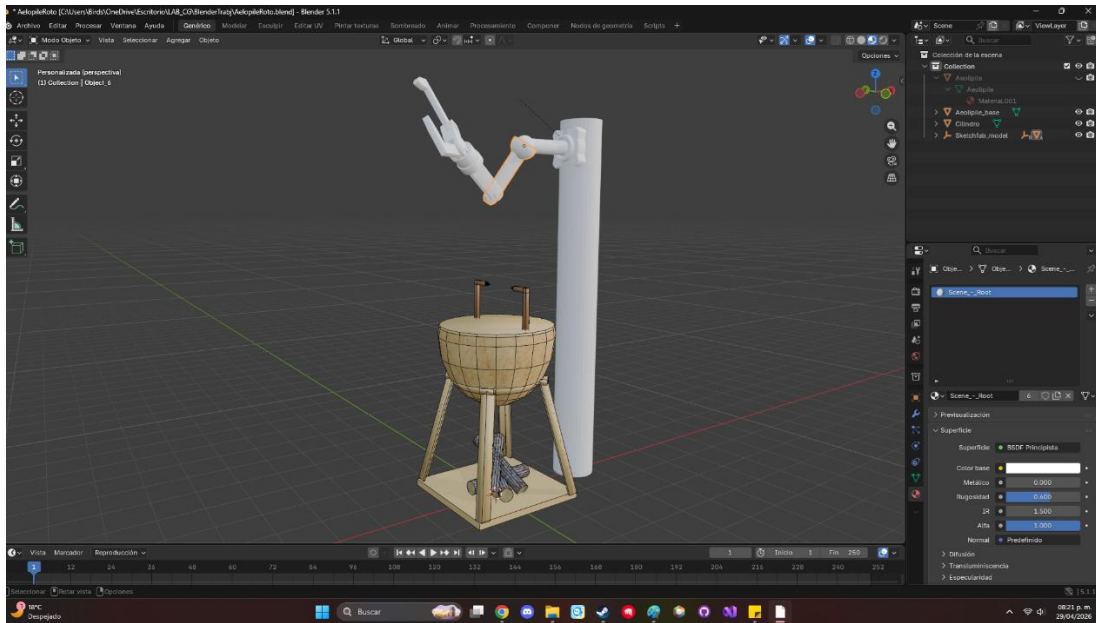
// 11. HUMO ANIMADO (Vinculado a la velocidad del motor) -----
// El humo solo existe si el Aeolipile esta girando aun
if (velAeolipile > 0.0f) {
    model = glm::mat4(1.0f);
    model = glm::translate(model, glm::vec3(0.0f, 4.0f, 20.0f));
    // Como velAeolipile va de 0.0 a 15.0, dividimos para obtener un porcentaje (0.0 a 1.0)
    escalaHumo = velAeolipile / 15.0f;
    // --- ANIMACIÓN FLOTANTE ---
    // Hacemos que el humo ondule hacia los lados usando senos y cosenos
    ondulacionX = sin(now * 2.0f) * 0.3f * escalaHumo;
    ondulacionZ = cos(now * 2.5f) * 0.3f * escalaHumo;
    // Movemos un poquito la base del humo para simular flote
    model = glm::translate(model, glm::vec3(ondulacionX, 0.0f, ondulacionZ));
    // Aplicamos la escala
    model = glm::scale(model, glm::vec3(4.5f * escalaHumo, 5.5f * escalaHumo, 4.5f * escalaHumo));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    // Limpiamos los offsets
    toffset = glm::vec2(0.0f, 0.0f);
    glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
    color = glm::vec3(1.0f, 1.0f, 1.0f);
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    Humo.UseTexture();
    Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
    meshList[3]->RenderMesh(); // Planos cruzados en X
}
}

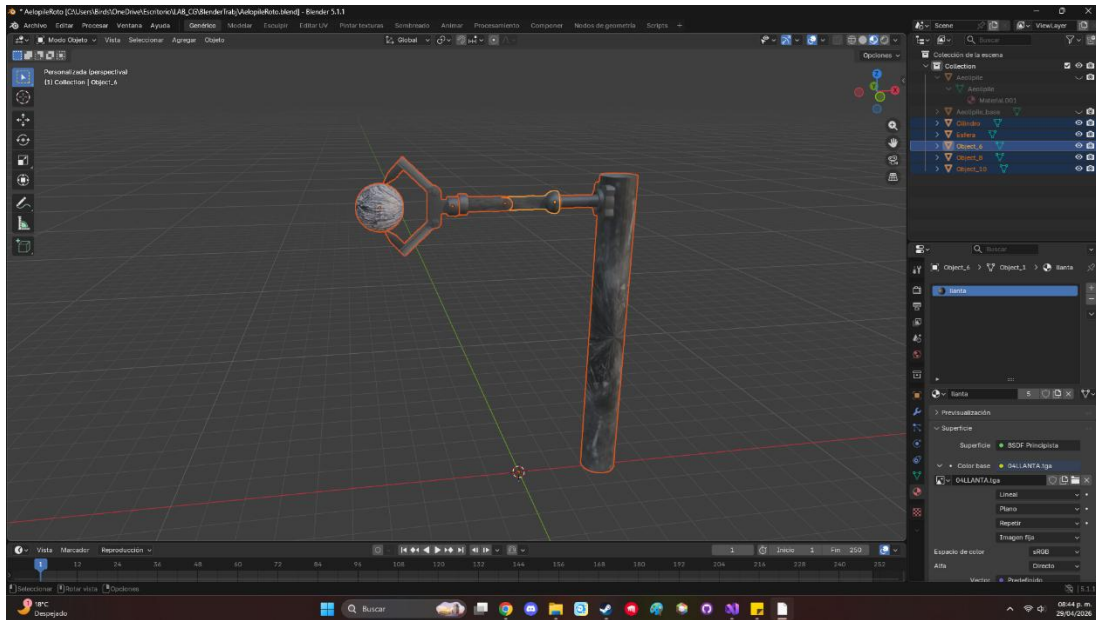
```

ACTIVIDAD 2. Brazo

Instrucciones: “Brazo/palanca que lance la esfera metálica. El humo crecido inicia el movimiento del brazo palanca. Animación del brazo para lanzar”

Para el brazo, comenzamos preparando un modelo en blender, este se descargó de [sketchfab](#) pero se modifico y texturizo por cuenta propia. Además, se enlazo con un “poste” para que el brazo no estuviera flotando en el aire en la visualización final. Cada una de las partes del brazo se guardo como .obj separados para poder jerarquizarlas más adelante y se coloco una esfera texturizada para la siguiente actividad.





Continuamos exportando nuestros modelos e instanciando multiples variables para la animación del brazo mecánico.

```
Model BrazoBaseT;
Model Brazo01T;
Model Brazo02T;
Model BrazoGT;
// --- Animación: Movimiento de brazo ---
float rotArticulacion1; // Rotación en Z entre la Base y el Brazo01
float rotArticulacion2; // Rotación en Z entre el Brazo01 y el Brazo02
float rotArticulacion3; // Rotación en X de la Garra
int estadoBrazo; // Máquina de estados: 0 (Reposo), 1 (Preparando), 2 (Lanzando), 3 (Regresando)
...
...
BrazoBaseT = Model();
Brazo01T = Model();
Brazo02T = Model();
BrazoGT = Model();
BrazoBaseT.LoadModel("Models/BrazoBaseT.obj");
Brazo01T.LoadModel("Models/Brazo01T.obj");
Brazo02T.LoadModel("Models/Brazo02T.obj");
BrazoGT.LoadModel("Models/BrazoGT.obj");
// --- Valores Iniciales: Movimiento de brazo ---
rotArticulacion1 = 0.0f; // Inicia en reposo
rotArticulacion2 = 0.0f; // Inicia en reposo
rotArticulacion3 = 0.0f; // Inicia en reposo
estadoBrazo = 0; // Arranca en estado 0 (esperando a que el humo crezca)
// --- Matrices: Brazo Mecánico y Jerarquía ---
glm::mat4 matrizBrazoBase; // Matriz de transformación para la base inmóvil del brazo
glm::mat4 matrizBrazo01; // Hereda transformaciones de la base
glm::mat4 matrizBrazo02; // Hereda transformaciones del Brazo01
glm::mat4 matrizGarra; // Hereda transformaciones del Brazo02
glm::mat4 matrizEsfera; // Hereda de la garra hasta ser lanzada independientemente
```

Dentro de nuestro int main declaramos la lógica de animación para el brazo. Como en ejercicios pasados, hacemos una máquina de estados finita controlada por la variable entera estadoBrazo, lo que nos permite dividir la animación en fases secuenciales. El ciclo empieza automáticamente cuando el Aeolipile alcanza su velocidad máxima ($velAeolipile \geq 15.0f$), asegurando de forma visual que el brazo actúe únicamente cuando el humo ha alcanzado su máximo volumen. En la Fase 1 (Preparación), las articulaciones rotan negativamente para simular que nuestro brazo mecánico está tomando impulso con topes de rotación (-20°, -60° y -90°). Cuando ya se preparó para lanzar, pasamos a la Fase 2 (Lanzamiento), donde se invierte la rotación con una velocidad

mucho mayor para ejecutar un movimiento rápido. Justo al terminar esta fase se activa la bandera esferaLanzada, se extraen las coordenadas globales exactas desde la matriz de transformación de la esfera (matrizEsfera[3]) para separarla de la jerarquía del brazo, y se le agrega un vector de impulso inicial (velE) para que inicie su propio cálculo de tiro parabólico. Finalmente, el código entra a la Fase 3 (Regreso), donde revertimos de manera más suave las rotaciones de las articulaciones hasta su origen (0.0f) y reiniciando la máquina de estados a 0, dejando el sistema en reposo a la espera del siguiente ciclo del Aeolipile.

```
// 7. Lógica del Brazo Mecánico (Máquina de Estados) -----
// DETONANTE PRINCIPAL (Segundo 12.5 / 625 unidades)
// -> Si el motor alcanzó su velocidad máxima (15.0f), el humo ya creció lo suficiente.
if (velAeolipile >= 15.0f && estadoBrazo == 0) {
    estadoBrazo = 1;
}
// MÁQUINA DE ESTADOS DEL MOVIMIENTO
if (estadoBrazo != 0) {
    // FASE 1: PREPARACIÓN Y TOMA DE IMPULSO (Del segundo 12.5 al 16.5)
    // -> Las articulaciones se hacen hacia atrás durante 200.0f unidades (4 seg?)
    if (estadoBrazo == 1) {
        rotArticulacion1 -= 0.1f * deltaTime; // Meta: -20° (Vel: 20° / 200 = 0.1f)
        rotArticulacion2 -= 0.3f * deltaTime; // Meta: -60° (Vel: 60° / 200 = 0.3f)
        rotArticulacion3 -= 0.45f * deltaTime; // Meta: -90° (Vel: 90° / 200 = 0.45f)
        // Topes individuales de seguridad para no exceder la pose de carga
        if (rotArticulacion1 <= -20.0f) rotArticulacion1 = -20.0f;
        if (rotArticulacion2 <= -60.0f) rotArticulacion2 = -60.0f;
        if (rotArticulacion3 <= -90.0f) rotArticulacion3 = -90.0f;
        // -> Si TODAS las piezas llegaron a su meta exacta, el brazo está cargado y listo.
        if (rotArticulacion1 == -20.0f && rotArticulacion2 == -60.0f && rotArticulacion3 ==
-90.0f) {
            estadoBrazo = 2;
        }
    }
    // FASE 2: LANZAMIENTO (Del segundo 16.5 al 17.5)
    // -> Movimiento rápido hacia adelante de 50.0f unidades (1 seg?)
    else if (estadoBrazo == 2) {
        rotArticulacion2 += 1.6f * deltaTime; // Avanza 80° (de -60° llega a +20°)
        rotArticulacion3 += 1.8f * deltaTime; // Avanza 90° (de -90° vuelve a 0°)
        // Topes de llegada al punto máximo de extensión de la pinza
        if (rotArticulacion2 >= 20.0f) rotArticulacion2 = 20.0f; // Se clava en 20°
        if (rotArticulacion3 >= 0.0f) rotArticulacion3 = 0.0f; // Vuelve a su origen de 0°
        // -> Cuando termina (exactamente en el segundo 17.5), liberamos la carga.
        if (rotArticulacion2 == 20.0f && rotArticulacion3 == 0.0f) {
            // LANZAMIENTO DE LA ESFERA
            esferaLanzada = true; // La esfera se moverá por "separado"
            tiempoVueloEsfera = 0.0f; // Reloj de vuelo independiente de la esfera
            // -> Extraemos la posición global (X,Y,Z) de la esfera
            posE_X = matrizEsfera[3][0];
            posE_Y = matrizEsfera[3][1];
            posE_Z = matrizEsfera[3][2];
            // -> Le aplicamos la inercia (vector de velocidad inicial)
            velE_X = -0.35f; // Un impulso horizontal corto
            velE_Y = 0.60f; // Un salto vertical pequeño (subirá unas 4 unidades)
            velE_Z = 0.0f; // Bloqueada en el eje Z para que viaje en línea recta
            estadoBrazo = 3; // Terminamos el lanzamiento
        }
    }
    // FASE 3: REGRESO A LA NORMALIDAD (Del segundo 17.5 al 22.5)
    // -> El brazo vuelve a reposo durante durante 250.0f unidades (5 seg?)
    else if (estadoBrazo == 3) {
        rotArticulacion1 += 0.08f * deltaTime; // De -20° a 0°
        rotArticulacion2 -= 0.08f * deltaTime; // De +20° a 0°
        // Topes de llegada al origen (0.0f)
        if (rotArticulacion1 >= 0.0f) rotArticulacion1 = 0.0f;
        if (rotArticulacion2 <= 0.0f) rotArticulacion2 = 0.0f;
        // -> Apagamos la máquina de estados dejándola en 0, lista para otro ciclo.
        if (rotArticulacion1 == 0.0f && rotArticulacion2 == 0.0f) {
            estadoBrazo = 0;
        }
    }
}
}
```

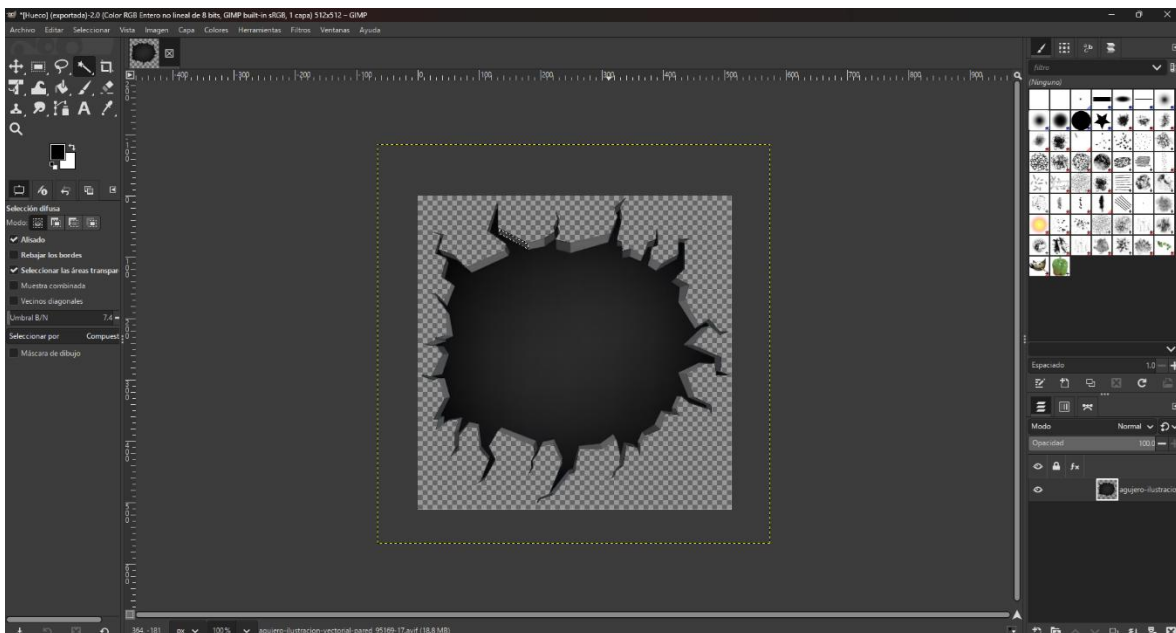
Finalmente, renderizamos nuestros modelos del brazo seccionado mediante jerarquía.

```
// BRAZO MECANICO (Jerarquía Cinemática Directa) -----  
// 1. EL PADRE: BrazoBaseT  
matrizBrazoBase = glm::mat4(1.0f);  
matrizBrazoBase = glm::translate(matrizBrazoBase, glm::vec3(2.5f, -2.0f, 20.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(matrizBrazoBase));  
BrazoBaseT.RenderModel();  
// Hereda la transformación de la Base  
matrizBrazo01 = matrizBrazoBase;  
matrizBrazo01 = glm::translate(matrizBrazo01, glm::vec3(-1.6f, 8.2f, 0.0f));  
matrizBrazo01 = glm::rotate(matrizBrazo01, rotArticulacion1 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(matrizBrazo01));  
Brazo01T.RenderModel();  
// Hereda la transformación de Brazo01T  
matrizBrazo02 = matrizBrazo01;  
matrizBrazo02 = glm::translate(matrizBrazo02, glm::vec3(-1.4f, 0.0f, 0.0f));  
matrizBrazo02 = glm::rotate(matrizBrazo02, rotArticulacion2 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(matrizBrazo02));  
Brazo02T.RenderModel();  
// Hereda la transformación de Brazo02T  
matrizGarra = matrizBrazo02;  
matrizGarra = glm::translate(matrizGarra, glm::vec3(-1.3f, 0.0f, 0.0f));  
matrizGarra = glm::rotate(matrizGarra, rotArticulacion3 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(matrizGarra));  
BrazoGT.RenderModel();
```

ACTIVIDAD 3. Esfera

Instrucciones: “Esfera metálica. Movimiento de la esfera metálica por medio de física básica: tiro parabólico y rebote. Al dar el último rebote la esfera entra en un "hueco" en el piso y reaparece en el brazo”

Preparamos con antelación una textura de hueco para el piso de nuestro entorno mediante GIMP y la descarga previa de una imagen de [magnific](#)



Finalmente exportamos nuestra textura y declaramos aun más variables para el tiro parabólico.

```
Texture Hueco;
Model EsferaT;
// --- Animación: Tiro Parabólico de la Esfera ---
bool esferaLanzada;           // Bandera lógica para romper la jerarquía del brazo y aplicar el tiro
bool detenerRebote;          // Bandera para iniciar la fase de rebote tras el primer impact
float posE_X;                 // Coordenada global en el eje X de la esfera durante el vuelo
float posE_Y;                 // Coordenada global en el eje Y de la esfera durante el vuelo
float posE_Z;                 // Coordenada global en el eje Z de la esfera durante el vuelo
float velE_X;                 // Vector de velocidad actual en el eje X
float velE_Y;                 // Vector de velocidad actual en el eje Y
float velE_Z;                 // Vector de velocidad actual en el eje Z
float gravedadE;              // Aceleración vertical hacia abajo (ajustada para dar peso a la esfera metálica)
float tiempoVueloEsfera;     // Cronómetro exclusivo para desaparecer la esfera tras 7.5 segundos

...

Hueco = Texture("Textures/Hueco.tga");
co.LoadTextureA();
EsferaT = Model();
EsferaT.LoadModel("Models/EsferaT.obj");

// --- Valores Iniciales: Tiro Parabólico de la Esfera ---
esferaLanzada = false;        // Inicia sujeta a la garra
detenerRebote = false;        // No hay rebote aun
posE_X = 0.0f;                // Origen temporal
posE_Y = 0.0f;                // Origen temporal
posE_Z = 0.0f;                // Origen temporal
velE_X = 0.0f;                // Inicia sin vel
velE_Y = 0.0f;                // Inicia sin vel
velE_Z = 0.0f;                // Inicia sin vel
gravedadE = -0.04f;           // Gravedad... al "doble"
tiempoVueloEsfera = 0.0f;     // Tiempo que lleva volando la esfera
```

Dentro de nuestras dos animaciones pasadas definimos si la esfera fue lanzada o no, además de inicializar su posición por separado para actuar como un objeto individual más allá de la jerarquía actual.

```
// ANIMACION AEOLIPILE
// ETAPA 5: REPOSO TOTAL Y REINICIO
else {
    // Si ya apagamos el fuego y el motor se detuvo por completo:
    // -> Regresamos el cronómetro a cero para dejar la máquina lista para otro ciclo.
    tiempoFuego = 0.0f;
    // -> Reiniciamos la bandera para devolver la esfera a su punto de origen.
    esferaLanzada = false; // LA ESFERA REAPARECE EN EL BRAZO
}

...

// ANIMACIÓN BRAZO
if (rotArticulacion2 == 20.0f && rotArticulacion3 == 0.0f) {
    // LANZAMIENTO DE LA ESFERA
    esferaLanzada = true;        // Aquí marcamos que la esfera se movera por "separado"
    tiempoVueloEsfera = 0.0f;    // Iniciamos el reloj de vuelo independiente de la esfera
    // -> Extraemos la posición global (X,Y,Z) de la última posición de la esfera
    posE_X = matrizEsfera[3][0];
    posE_Y = matrizEsfera[3][1];
    posE_Z = matrizEsfera[3][2];
    // -> Le aplicamos la inercia (vector de velocidad inicial)
    velE_X = -0.35f;             // Un impulso horizontal corto para que caiga dentro de la zona
    velE_Y = 0.60f;             // Un salto vertical pequeño (subirá unas 4 unidades)
    velE_Z = 0.0f;              // Bloqueada en el eje Z para que viaje en línea recta
    estadoBrazo = 3;             // Terminamos el lanzamiento
}
```

Para simular el movimiento independiente de la esfera, usamos el tiro parabólico como se pidió, este se logra aplicando una fuerza de gravedad constante (gravedadE) que disminuye continuamente la velocidad vertical (velE_Y), atrayendo la esfera hacia el suelo de forma natural mientras su mov en el eje X la hace avanzar hacia adelante. Para darle realismo, programamos un sistema de colisiones: cada vez que la esfera choca con el piso (-1.5f), su dirección vertical se invierte (generando el rebote) y pierde un 35% de su energía, al multiplicar la velocidad vertical

por el factor decimal 0.65f al momento del impacto, lo que significa que la esfera solo conserva el 65% de su fuerza original tras cada rebote, al mismo tiempo que una fricción horizontal (en nuestro eje X) que hace que nuestra esfera vaya frenando. Por último, utilizamos la función de valor absoluto para detectar el momento exacto en el que la esfera pierde todo su movimiento al lanzarse ($velE_X < 0.01f$) y al cumplirse esta condición, terminamos el rebote y obligamos a la esfera a caer lentamente en línea recta por el eje Y, creando la ilusión de que cae y se hunde dentro del hueco de nuestro escenario.

```
// Esfera (Ultima en la jerarquía, es algo más independiente) -----
// REPOSO la esfera hereda la transformación de garra
if (!esferalanzada) {
    matrizEsfera = matrizGarra;
    matrizEsfera = glm::translate(matrizEsfera, glm::vec3(-2.0f, 0.0f, 0.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(matrizEsfera));
    EsferaT.RenderModel();
}
// VUELO
else {
    // Sumamos el tiempo real a su cronómetro de vuelo exclusivo
    tiempoVueloEsfera += deltaTime;
    // TIEMPO DE VISUALIZACIÓN DE LA ESFERA 375.0f unidades (7.5 seg?)
    if (tiempoVueloEsfera < 375.0f) {

        // Tiro parabolico
        // -> La gravedad resta velocidad en el eje Y constantemente (atracción al suelo)
        velE_Y += gravedadE * deltaTime;
        // -> El vector de velocidad actual modifica la posición global
        posE_X += velE_X * deltaTime;
        posE_Y += velE_Y * deltaTime;
        posE_Z += velE_Z * deltaTime; // Seguirá siendo 0.0f
        // -> Usamos el valor abs de velX para verificar si la esfera dejo de rodar
        detenerRebote = (std::abs(velE_X) < 0.01f);
        // REBOTE Y FRICCIÓN
        // -> Si choca con el nivel del piso (-1.5f) y aún no se detiene por completo:
        if (posE_Y <= -1.5f && !detenerRebote) {
            posE_Y = -1.5f; // Clavamos la posición para que no atravesase el suelo sólido
            velE_Y = -velE_Y * 0.65f; // Rebota: Invierte dirección vertical y pierde 35% de F
            velE_X *= 0.85f; // Fricción: Pierde el 15% de su velocidad horizontal por el choque
        }

        // HUNDIMIENTO
        else if (detenerRebote) {
            velE_X = 0.0f; // Bloqueamos el eje X para asegurar que baje perfectamente recta
            velE_Y = 0.0f; // Anulamos la gravedad acumulada para que no caiga como una bala
            // -> Hacemos que se hunda
            posE_Y -= 0.05f * deltaTime;
            // Tope de profundidad:
            if (posE_Y <= -7.5f) {
                posE_Y = -7.5f;
            }
        }

        // RENDERIZADO INDEPENDIENTE DE LA ESFERA EN VUELO
        matrizEsfera = glm::mat4(1.0f);
        matrizEsfera = glm::translate(matrizEsfera, glm::vec3(posE_X, posE_Y, posE_Z));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(matrizEsfera));
        EsferaT.RenderModel();
    }
}
```

EJECUCIÓN

Para visualizar todo el proceso de animación tenemos la **tecla L**



Se adjunta link de drive a un vídeo con la ejecución completa.

<https://drive.google.com/file/d/1aMDkar1yd75SNptwCFR0TcqUJEcQGnJR/view?usp=sharing>

PROBLEMAS PRESENTADOS

Para la actividad 1 y 2 no se presentaron problemas, pero para el desarrollo de la 3 si hubo un par de complicaciones con la implementación del tiro parabólico.

Atenuación infinita y "micro rebotes":

- Al aplicar las físicas de pérdida de energía (multiplicando la velocidad en Y por 0.65f tras cada impacto), la esfera sufría de micro rebotes infinitos y rápidos cuando la velocidad se acercaba a cero, impidiendo que se detuviera por completo de forma natural.
- Solución: Se desarrolló lo más parecido a un sensor lógico evaluando el valor absoluto de la velocidad horizontal ($\text{std::abs}(\text{velE_X}) < 0.01f$). Esto permitió establecer tolerancia; una vez que cruzábamos este límite mínimo, implementamos que las velocidades fueran forzosamente 0.0f, eliminando la vibración y estabilizando el modelo de la esfera.

Sincronización del hundimiento en el escenario:

- Originalmente, planear que la esfera cayera exactamente en el "hueco" del piso requería depender de una coordenada estática en el eje X. Si se modificaba el impulso inicial, la esfera terminaba hundiéndose en zonas donde el piso era sólido, viéndose extraño.

- Solución: En lugar de usar una coordenada X rígida, se vinculó el hundimiento a la pérdida de inercia previamente mencionada. Al detectar que la esfera se detenía por fricción, el sistema pudo finalizar con la colisión del piso y comenzaba a restar unidades progresivamente en el eje Y ($\text{posE_Y} -= 0.05f * \text{deltaTime};$). Así, la textura del hueco simplemente se colocó en la coordenada donde se detenía nuestra esfera, logrando que el hundimiento fuera en la coordenada correcta, aunque debemos decir que la animación no quedo como idealmente esperábamos.

CONCLUSIONES

Sobre los ejercicios. La complejidad de esta práctica fue alta y algo tediosa debido al control de las distintas etapas de animaciones que implementamos y su control secuencia. Las actividades nos exigieron la integración de varios sistemas en simultáneo: en primer lugar, el uso de funciones trigonométricas y porcentajes dinámicos para animar el escalado del humo basándonos en la velocidad del Aeolipile. Posteriormente, el diseño lógico de una máquina de estados finita para administrar las transiciones mecánicas del brazo jerarquizado de forma ordenada. Finalmente, el mayor nivel de complejidad se presentó en la esfera, donde tuvimos que “romper” su dependencia jerárquica para programar un tiro parabólico, atenuación de sus rebotes y el cómo se detenía en el piso. Todo esto sincronizado a través del tiempo real (deltaTime).

Comentarios generales. La parte de la implementación de físicas (específicamente la matemática detrás de la pérdida de energía en los rebotes y el cálculo del tiro parabólico) faltó explicar a detalle algunos conceptos, aunque se entiende que era más cosa de nuestra investigación previa, habría sido de mucha ayuda que se nos proporcionará más documentación útil.

Cierre. En conclusión, con las actividades realizadas podemos afirmar que hemos pasado de una simple manipulación de vértices y texturas; a un intento más detallado de la simulación de leyes físicas para dotar a los modelos de lo que nosotros en el mundo real conocemos como un “peso, inercia y realismo”. Lograr que una esfera metálica responda a la gravedad y pierda energía de forma congruente, o que el humo oscile mediante trigonometría en función de la fricción del motor, nos requirió un análisis matemático más tardado que el propio código en sí.

REFERENCIAS

- Bourg, D. M., & Bywalec, B. (2013). *Physics for Game Developers: Science, math, and code for realistic effects* (2nd ed.). O'Reilly Media.
- Fiedler, G. (2004). *Integration Basics*. Gaffer On Games. Recuperado de https://gafferongames.com/post/integration_basics/
- Magnific. (s.f.). *Agujero ilustración vectorial pared* [Imagen en formato vector]. Recuperado de https://www.magnific.com/es/vector-premium/agujero-ilustracion-vectorial-pared_4191106.htm
- Sketchfab. (s.f.). *Robot arm blender mechanically rigged* [Modelo 3D]. Recuperado de <https://sketchfab.com/3d-models/robot-arm-blender-mechanically-rigged-6fa4d326c5314b71be3e6869ce7b9c9f>